Assignment 4 – CO4

Inheritance, Class Hierarchies and Pointers to Objects

Field	Details
Name / Register No.	Kaviya V/192421022
GitHub repository	https://github.com/kaviyavellaisamy/DSA0107---C-/tree/main

All code compiles with g++ -std=c++17 -Wall -Wextra. Sanitizer runs add -fsanitize=address (and undefined). Pseudocode uses the structured layout ALGORITHM / INPUT / OUTPUT / BEGIN … END with ← for assignment and upper-case keywords; adjust the headings to match your copy of the SIMATS Engineering – Standard for Algorithmic Pseudocode if it differs. Progression: Interpret → Identify Is-a / Has-a → Design → Decide access → Implement → Trace construction → Challenge → Refine.

Task 1 – Class Design Review: Diagnose and Repair Duplicated Classes (6 marks)

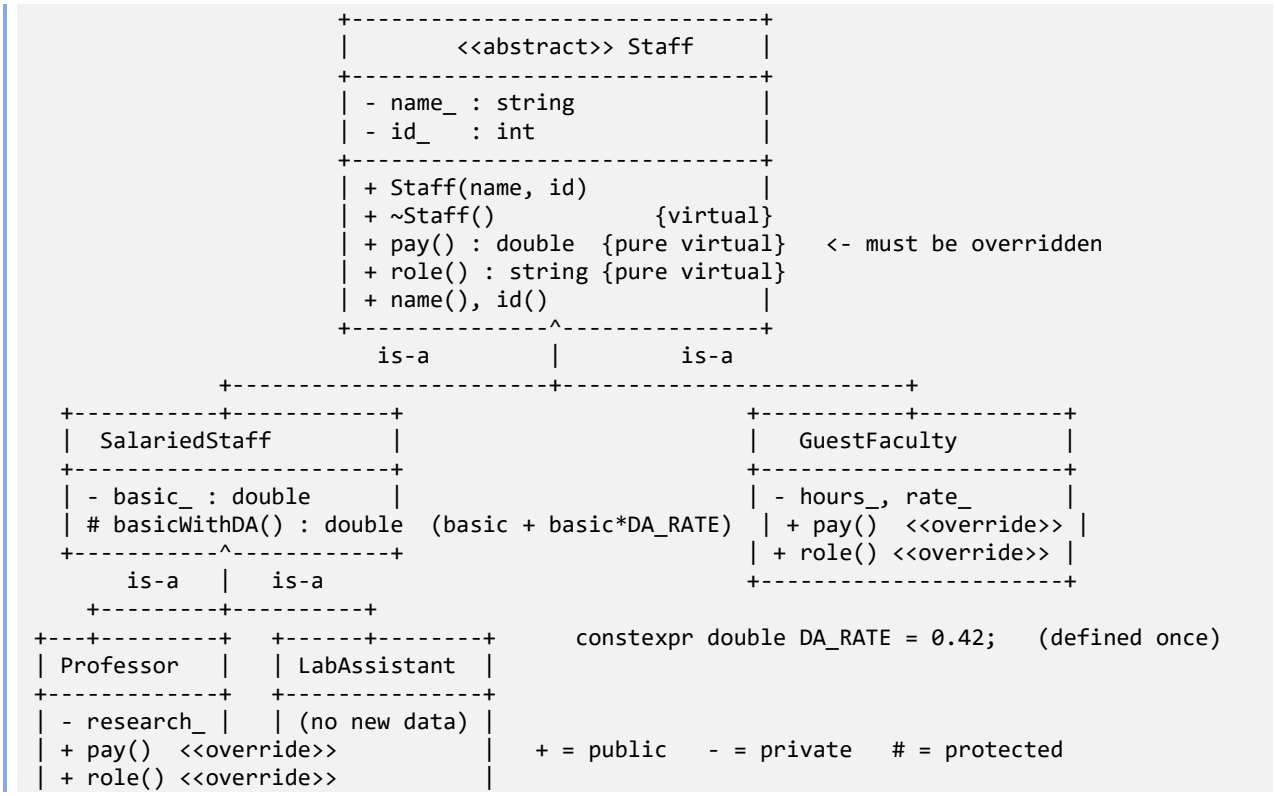
Task 1.1 – Diagnose the design (2 marks)

#	Deficiency in the original code	Consequence
1	Repeated fields (name, id, basic, da) and a repeated pay rule (basic + basic * 0.42) in Professor and LabAssistant	A bug fixed in one class stays in the other; code grows with every category
2	All data members are public	Any code can set basic = -5 or change id; no validation is possible
3	Inconsistent names for the same information: GuestFaculty uses nm and code where the others use name and id	Generic code cannot read the name or the id of every staff member uniformly
4	The field da is declared in two classes but never used – the formula uses the literal 0.42 instead	Dead data: misleading, and 8 wasted bytes per object
5	The magic number 0.42 appears twice	A change in the DA rate must be found and edited in every copy
6	No constructors: fields hold garbage until each is assigned	Objects can be used half-initialised; pay() may read uninitialised values
7	No common base type: a Professor, a LabAssistant and a GuestFaculty cannot be stored in one array or processed in one loop	The payroll needs one loop per class
8	Adding a new staff category means copying a class and editing every loop that processes staff	Effort grows with each category; each edit risks breaking working code

Evidence: compiling the original three classes shows sizeof = 64, 56 and 56 bytes, and a program that wants a total payroll has to call p.pay() + l.pay() + g.pay() by name (printed 286 for my test values); there is no way to write for (auto& s : staff).

Task 1.2 – Repair the design (3 marks)

UML class diagram




```
constexpr double DA_RATE = 0.42;                        // one place
class Staff {                                           // abstract base
    std::string name_; int id_;
public:
    Staff(const std::string& n, int id) : name_(n), id_(id) {}
    virtual ~Staff() = default;
    virtual double pay() const = 0;
    virtual std::string role() const = 0;
};
class SalariedStaff : public Staff {                    // basic + DA, shared by two classes
    double basic_;
protected:
    double basicWithDA() const { return basic_ + basic_ * DA_RATE; }
public:  SalariedStaff(const std::string& n, int id, double b) : Staff(n, id), basic_(b) {}
};
class Professor    : public SalariedStaff { double research_; public: /* ctor */ double pay()
const override { return basicWithDA() + research_; } /* role */ };
class LabAssistant : public SalariedStaff { public: using SalariedStaff::SalariedStaff; double
pay() const override { return basicWithDA(); } /* role */ };
class GuestFaculty : public Staff { double hours_, rate_; public: /* ctor */ double pay()
const override { return hours_ * rate_; } /* role */ };
```

Actual output (all staff processed in ONE loop through Staff*, run under -fsanitize=address, no errors)

```
Professor      Dr. Rao     118600.00
LabAssistant   Meena        42600.00
GuestFaculty   Mr. Iyer     30000.00
Total payroll 191200.00   (hand check: 118600 + 42600 + 30000 = 191200)
```

Hand check: Professor 80000 + 80000 × 0.42 + 5000 = 118,600; LabAssistant 30000 + 30000 × 0.42 = 42,600; GuestFaculty 20 h × 1500 = 30,000.

Task 1.3 – Design defence (1 mark, 98 words)

A new staff category is one new class: derive from Staff (or SalariedStaff), write its constructor and override pay() and role(). Nothing existing is edited, because the payroll loop works on Staff pointers and calls pay() through the virtual table. The shared DA rule lives in one constant and the common data (name, ID) is declared once in the base class. Before, a new category meant copying a class and editing every loop and report. Now the compiler also forces the new class to implement pay(), since it is pure virtual, so a pay rule cannot be forgotten.

Alternative concept – one Staff class with a type field and a switch in pay()

```
struct StaffFlat { int type; string name; int id; double basic, da, research, hours, rate;
    double pay() const { switch (type) { case 0: return basic + basic*DA_RATE + research;
                                         case 1: return basic + basic*DA_RATE;
                                         case 2: return hours * rate; } return 0; } };
```

Measure	Hierarchy (virtual pay())	Single class + switch
Time per pay() call	O(1): one indirect call through the vtable	O(1): one switch (compare/jump table)
Space per object (measured sizeof)	Professor 64, LabAssistant 56, GuestFaculty 64 bytes (includes an 8-byte vtable pointer)	88 bytes for EVERY object: the unused fields (hours, rate for a professor, research for an assistant) are carried by all
Places that change to add a category	1: one new class; no existing code edited	At least 4: new enum/type value, the switch in pay(), the switch in role() (and any report or tax switch), the constructor
Compiler help if a case is forgotten	Pure virtual forces pay() to be written	Only a -Wswitch warning, and only if an enum is used

Task 2 – From Requirement to Hierarchy: University People Management (8 marks)

Task 2.1 – Identify the relationships (2 marks)

Relationship	Is-a or has-a	Type of inheritance
Student is a Person	is-a	Single (Student → Person)
Staff is a Person	is-a	Single; together with the line above, Student and Staff derived from Person form a hierarchical structure
TeachingAssistant is both a Student and a Staff member	is-a (twice)	Multiple (TA → Student, Staff); with the shared Person this is the diamond, i.e. hybrid inheritance

Relationship	Is-a or has-a	Type of inheritance
ResearchScholar is a Student who also holds a fellowship	is-a Student; the fellowship amount is a data member	Single, and multilevel as a chain ResearchScholar → Student → Person
Person has an Address (city, pin)	has-a	Composition (member class)
Student has a Course	has-a	Composition (member class)

Task 2.2 – Resolve the diamond (2 marks)

```
           Person
          /      \
     Student    Staff            without "virtual": TeachingAssistant contains TWO Person
sub-objects
          \      /
     TeachingAssistant
```

The problem. If Student and Staff each inherit Person normally, a TeachingAssistant contains two complete Person sub-objects (one inside Student, one inside Staff). Then ta.id is ambiguous: g++ reports **error: request for member 'id' is ambiguous … candidates are: 'int Person::id'**; the name, id and phone are stored twice and the two copies can disagree (a TA with two different phone numbers). **The solution.** Declare Student and Staff with virtual public Person. Then every TeachingAssistant has exactly one shared Person sub-object, which is constructed once by the most-derived class. Measured with real data members (name, id, phone in Person): non-virtual TeachingAssistant = 224 bytes; virtual = 168 bytes.

Task 2.3 – Design the classes (3 marks)

```
class Address { std::string city; int pin; public: Address(const std::string& c, int p); };
// has-a
class Course  { std::string code; public: explicit Course(const std::string& c); };
// has-a
class Person {
protected: std::string name; int id; std::string phone; Address addr;      // protected:
derived classes may use
public:    Person(const std::string& n, int id, const std::string& ph, const Address& a);
           virtual ~Person() = default;
};
class Student : virtual public Person  { std::string programme; double cgpa; Course course; /*
... */ };
class Staff   : virtual public Person  { std::string department; double salary; /* ... */ };
class TeachingAssistant : public Student, public Staff { /* ... */ };
class ResearchScholar   : public Student { double fellowship; /* ... */ };
```

Base class	Inherited as	Reason
Person → Student	public (virtual)	A student IS a person; Person's public interface (name, id, phone) must stay public on a Student so that code written for Person works on Student objects (substitutability).
Person → Staff	public (virtual)	Same is-a reason; virtual to keep one Person in a TeachingAssistant.

Base class	Inherited as	Reason
Student, Staff → TeachingAssistant	public, public	A TA IS a student and IS a staff member; both interfaces are available to callers.
Student → ResearchScholar	public	A scholar is a student with a fellowship.
private or protected inheritance	not used	They would model "implemented in terms of", not "is-a": the base interface would be hidden from users of the derived object.

Task 2.4 – Access table (1 mark)

Visibility, inside Student, of the members of Person under the three kinds of inheritance. (private members of Person are never directly accessible in Student; they exist in the object but are reachable only through Person's own member functions.)

Member of Person	public inheritance	protected inheritance	private inheritance
public	public in Student	protected in Student	private in Student
protected	protected in Student	protected in Student	private in Student
private	not accessible	not accessible	not accessible

Predict the Output

Prediction (hand-traced) and actual output of the given program

```
Person Student Staff TA
```

Why Person is printed only once. Student and Staff inherit Person virtually. The virtual base is constructed first and only once, by the most-derived class (TeachingAssistant); the calls to Person from Student and Staff are skipped. Then the direct bases are constructed in declaration order (Student, then Staff), then the body of TeachingAssistant prints TA.

Experiment: remove the word virtual from both base classes

```
Prediction:  Person Student Person Staff TA      (each base builds its own Person)
Actual    :  Person Student Person Staff TA      (identical; compiled with -Wall -Wextra: no
warnings)
sizeof(TeachingAssistant) = 16 with virtual, 2 without   (empty classes: with virtual, two
vptrs; without, two 1-byte Person copies)
```

Alternative concept – composition: TeachingAssistant has-a Student and has-a Staff

Measure	Virtual inheritance	Composition (members)
Space per object (real data, measured)	168 bytes: one Person + 2 vptrs + class data	224 bytes: two full Person copies (same as non-virtual inheritance)
Reaching the name	ta.name: one unambiguous name; an extra indirection through the virtual-base offset (O(1))	ta.student.name and ta.staff.name: two names that must be kept consistent; direct offset (O(1))

Measure	Virtual inheritance	Composition (members)
Substitutability	A TA can be passed wherever a Person, Student or Staff is expected	A TA is not a Student or Staff; code taking Student& cannot accept it
Ambiguity	None (one Person)	None syntactically, but two sets of data (duplicate phone numbers)

Task 3 – Constructors in Derived Classes and Member Classes (9 marks)

Task 3.1 – Pass values to base constructors (3 marks)

```
Person(const string& n, const string& city, int pin) : name(n), addr(city, pin) {}
Student(const string& n, const string& city, int pin, const string& cc)
    : Person(n, city, pin), course(cc) {}
Staff(const string& n, const string& city, int pin, double s)
    : Person(n, city, pin), salary(s) {}
TeachingAssistant(const string& n, const string& city, int pin, const string& cc, double s)
    : Person(n, city, pin),                   // virtual base: the MOST-DERIVED class calls
it directly
      Student(n, city, pin, cc), Staff(n, city, pin, s) {}
```

Why the most-derived class must call the virtual base constructor: both Student and Staff would otherwise try to construct the single shared Person with their own arguments, and the compiler cannot decide whose to use. The rule is that the most-derived class initialises every virtual base directly, once; the Person(...) calls written in Student and Staff are ignored when a TeachingAssistant is built (but are still used when a plain Student or Staff is built). Since Person has no default constructor, TeachingAssistant must supply the arguments itself.

Task 3.2 – Member classes (2 marks)

Person has an Address member and Student has a Course member. Their constructors receive values through the initializer list of the enclosing constructor: Person(...) : name(n), addr(city, pin); and Student(...) : Person(...), course(cc). Members are constructed after the base classes and before the body of the constructor.

Task 3.3 – Trace the order (3 marks)

Actual output (every constructor and destructor prints its name; run under -fsanitize=address)

```
Student object    : Address(Salem) Person(Kavya) Course(CS101) Student | ~Student ~Course
~Person ~Address
Staff object      : Address(Chennai) Person(Ravi) Staff | ~Staff ~Person ~Address
TeachingAssistant : Address(Madurai) Person(Asha) Course(CS201) Student Staff TA | ~TA ~Staff
~Student ~Course ~Person ~Address
```

Step	Rule observed
Construction	1. virtual base(s) first (Person, whose own Address member is built just before Person's body runs); 2. direct base classes in the order they are DECLARED (Student then Staff); 3. each class's member objects in declaration order (Course); 4. finally the body of the derived constructor.

Step	Rule observed
Destruction	Exactly the reverse: the body of the derived destructor, then its members, then the base classes in reverse order, the virtual base last.
Why	A base class must exist before the derived part that depends on it, and must be destroyed after it.

Predict the Output – Car / Engine / Vehicle

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Rule: base class first (Vehicle), then the member object (Engine) in declaration order, then the body of Car; destruction in the reverse order: ~Car body, then member ~Engine, then base ~Vehicle. The | is printed while the object is alive, inside the inner block, and the destructors run at the closing brace.

Complete the Code – TODO 1 (Student constructor)

```
Student(const std::string& n, int a, long long r, const std::string& c, int pin)
    : Person(n, a), regNo(r), home(c, pin) {}
```

```
Actual output:  192311001 Kavya (19) Salem-636001           (matches the expected output)
```

Swapping the order of the initializer list

```
Student(...) : home(c, pin), regNo(r), Person(n, a) {}
g++ -Wall:  warning: 'Student::home' will be initialized after [-Wreorder]
            warning: 'long long int Student::regNo' ...  when initialized here
```

The warning says the written order is ignored: the real order is Person (base), regNo, home (declaration order). It matters when one member's initialiser uses another member.

Task 3.4 – Experiment: remove virtual (1 mark)

	With virtual	Without virtual
Output (given program)	Person Student Staff TA	Person Student Person Staff TA
sizeof(TeachingAssistant) (given empty classes)	16	2
sizeof with real data (name, id, phone in Person)	168	224

Alternative concept – Student holds Address* created with new

```
class StudentP { Address* home;
public:
    StudentP(const string& c, int p) : home(new Address(c, p)) {}
    ~StudentP() { delete home; }                                                    // must
release it
    StudentP(const StudentP& o) : home(new Address(*o.home)) {}                     // must
deep copy
    StudentP& operator=(const StudentP& o) { if (this != &o) { Address* t = new
Address(*o.home); delete home; home = t; } return *this; }
```

```
};
```

Extra time: one heap allocation per Student (O(1), typically tens of nanoseconds) plus a pointer dereference on every access. Extra space: 8 bytes for the pointer (Student with a member object: 40 bytes; with a pointer: 8 bytes + a separate 40-byte block + allocator overhead). The destructor must delete the Address; the copy constructor must allocate and copy a new Address (otherwise two Students share one Address and delete it twice); the copy assignment must do the same with a self-assignment guard – the Rule of Three that a plain member object gives for free.

Task 4 – Pointers to Derived Classes: Forensics (7 marks)

Task 4.1 – Locate the defects (2 marks)

Defect	Cause	Effect
The senior rate (4.0) is never used	rate() is not virtual, so a->rate() through an Account* is bound at compile time to Account::rate	Both accounts print 3.5
Memory is never released	Account's destructor is not virtual: delete a on an Account* runs only ~Account; ~SeniorAccount (which does delete[] history) never runs	The 800-byte history array leaks; formally the behaviour is undefined
Object slicing in the last line	Account copy = SeniorAccount(); copies only the Account sub-object into an Account; the SeniorAccount part (history pointer, overridden rate) is cut off	A vector<Account> holds plain Accounts: rate() gives 3.5, and sizeof(Account) = 1 versus sizeof(SeniorAccount) = 8

Task 4.2 – Construct evidence (2 marks)

Original program, no sanitizer (destructors print their names)

```
3.5
3.5                                                <- senior account should print 4
~Account ~Account                                  <- two deletes: ~SeniorAccount never ran
~SeniorAccount ~Account                            <- the temporary in
v.push_back(SeniorAccount()) (this is the sliced copy being made)
stored in vector<Account>: rate = 3.5  sizeof(Account)=1 sizeof(SeniorAccount)=8
~Account
```

Original program under -fsanitize=address

```
ERROR: AddressSanitizer: new-delete-type-mismatch
  size of the allocated type:   8 bytes;
  size of the deallocated type: 1 bytes.
```

The sanitizer notices that an 8-byte SeniorAccount was released as a 1-byte Account – the symptom of a missing virtual destructor. With that check switched off (ASAN_OPTIONS=new_delete_type_mismatch=0) the leak itself is reported: **LeakSanitizer: Direct leak of 800 byte(s) in 1 object(s)** (100 doubles = the history array).

Task 4.3 – Repair the program (2 marks)

```
class Account { public:
    virtual ~Account() { std::cout << "~Account "; }              // virtual destructor
    virtual double rate() const { return 3.5; } };
class SeniorAccount : public Account { double* history = new double[100];
public:
    ~SeniorAccount() override { delete[] history; std::cout << "~SeniorAccount "; }
    double rate() const override { return 4.0; } };                // override: checked by the
compiler
...
Account* accounts[2] = { new Account, new SeniorAccount };
for (Account* a : accounts) std::cout << a->rate() << "\n";       // run-time binding
for (Account* a : accounts) delete a;                              // runs the right
destructor chain
std::vector<std::unique_ptr<Account>> v;  v.push_back(std::make_unique<SeniorAccount>());   //
pointers, not copies
```

```
Actual output (ASan + LeakSanitizer: no errors, exit code 0):
3.5
4
~Account | ~SeniorAccount ~Account |
stored through unique_ptr: rate = 4
~SeniorAccount ~Account
```

Task 4.4 – Prevent recurrence (1 mark)

1. A class that is used as a base class (has virtual functions, or is deleted through a base pointer) must declare a virtual destructor.
2. Declare every overriding function with override, and make functions that vary by type virtual in the base; the compiler then reports a mismatch instead of silently hiding the base function.
3. (Also) Keep polymorphic objects in containers as pointers or references (or smart pointers), never by value, so no copy can slice them.

Predict the Output

Prediction and actual output

```
3.5 4 3.5
```

Value	Binding	Explanation
p->rate() = 3.5	compile time (early)	rate() is not virtual, so the call is decided by the static type of p, Account*: Account::rate.
p->vrate() = 4	run time (late)	vrate() is virtual: the call goes through the object's vtable to SeniorAccount::vrate.
copy.vrate() = 3.5	compile time in effect	Account copy = s; copies only the Account part of s (slicing). copy IS an Account object, its vptr points to Account's table, so even the virtual call gives Account::vrate = 3.5.

Alternative concept – std::vector<std::unique_ptr<Account>>

Time and space are the same order: O(n) to process n accounts, and a unique_ptr is the size of one raw pointer (8 bytes) with no extra cost for deletion. Which of the three defects can still occur? (1) The **leak caused by a forgotten delete** is gone: the vector destroys the objects automatically. (2) The **non-virtual destructor defect remains**: unique_ptr<Account> deletes through an Account*, and I measured the same new-delete-type-mismatch error with a non-virtual destructor. (3) The **non-virtual rate()** defect remains (the senior rate is still not used), and **slicing** is still possible if someone copies an Account by value. Smart pointers fix ownership, not the class design.

Task 5 – Pointers to Objects and the this Pointer (7 marks)

Task 5.1 – Four ways to reach an object (3 marks)

```
Cart a;      a.add("Rice", 60).add("Dal", 120).setDiscount(10);   // (a) directly by name
Cart* b = &a; b->add("Oil", 180);                                  // (b) through a pointer
with ->
void viaReference(Cart& c) { c.add("Salt", 20); }  viaReference(a); // (c) through a reference
Cart* d = new Cart;  d->add("Tea", 100).setDiscount(5);  delete d;  // (d) new, released with
delete

Cart& add(const std::string& item, double price) { ...; return *this; }         // *this
enables chaining
Cart& setDiscount(double discount) { this->discount = discount; return *this; } // this-> :
member vs parameter
```

Actual output

```
(a) direct by name     : 162.00
(b) pointer with ->    : 324.00   (same object: 1)
(c) through a reference: 342.00  (items now 4)
(d) new / delete       : 95.00
```

Hand check: (60 + 120) × 0.90 = 162; adding Oil: 360 × 0.90 = 324; adding Salt: 380 × 0.90 = 342; Tea: 100 × 0.95 = 95.

Task 5.2 – Change the requirement: array of pointers (2 marks)

```
Cart* active[4] = { &carts[0], &carts[1], &carts[2], &carts[3] };
Cart** best = active;                                                    // pointer
arithmetic
for (Cart** q = active + 1; q < active + 4; ++q) if ((*q)->total() > (*best)->total()) best =
q;
int bi = 0;                                                                // indexing
for (int i = 1; i < 4; ++i) if (active[i]->total() > active[bi]->total()) bi = i;
```

Actual: both methods give index 1 with total 600.00 (carts: 120, 600, 300 and 80 × 50 % = 40). The loop is O(n) and needs no copies of the carts.

Task 5.3 – Compare (2 marks)

Way	Safety	Lifetime	When to use
Direct (by name)	Safest: cannot be null or dangling	Automatic: until the end of the scope	Default choice for objects whose lifetime matches a scope
Pointer with ->	Can be null, uninitialised, or dangling after the object dies	Does not control the lifetime of the object it points to	Optional or re-seatable access; arrays of pointers to existing objects
Reference	Cannot be null and cannot be rebound; dangling only if the object dies first	Same as the object it names	Function parameters (Cart&, const Cart&) and return values for chaining
new / delete	Leaks if delete is forgotten or an exception intervenes; double delete if repeated	Dynamic: until delete	When the lifetime is not tied to a scope; today prefer unique_ptr / make_unique

Why this is a pointer and what *this refers to: inside a non-static member function the compiler passes a hidden first argument, the address of the object on which the function was called, named this (type Cart* in a normal member, const Cart* in a const member). It is a pointer because it holds an address, so this->discount or *this is needed to reach the object; *this is the object itself. Returning *this by reference (Cart&) returns that same object, which is why calls can be chaine**

Complete the Code – TODO 1 and TODO 2

```
Cart& add(const std::string& item, double price)
{
    if (n < 10) { items[n] = item; prices[n] = price; ++n; }       // only if there is space
    return *this;                                                   // so calls can be chained
}
Cart& setDiscount(double discount)
{
    this->discount = discount;                                      // member = parameter
    return *this;
}
```

```
Actual output: 324.00        (matches expected output)
```

What if add() returned Cart instead of Cart&? Each call would return a COPY of the cart (416 bytes). The next call in the chain, .add("Dal", 120), would modify that temporary copy, not c. In my measurement the by-value version gave a total of 240.00 instead of 324.00: only Rice and Oil reached the real cart, and the discount set at the end of the chain was applied to a temporary that was thrown away. It is also slower (a 416-byte copy per call).

Alternative concept – std::vector<Cart> (objects) instead of an array of pointers

Measure	Array/vector of Cart*	vector<Cart>
Time to find the highest total (200,000 carts, -O2, measured)	O(n): 2.35 ms (extra pointer load per element)	O(n): 1.43 ms (carts contiguous)

Measure	Array/vector of Cart*	vector<Cart>
Space	8 bytes per pointer = 1.6 MB, plus the carts themselves	Carts copied into the vector: 416 bytes each = 83.2 MB
Pointers into the container when it grows	The pointed-to carts do not move	A pointer to an element is invalid after growth – measured: after 1000 push_backs the address had changed ("NO – dangling")

Task 6 – Design Challenge: Vehicle Rental Fleet (13 marks)

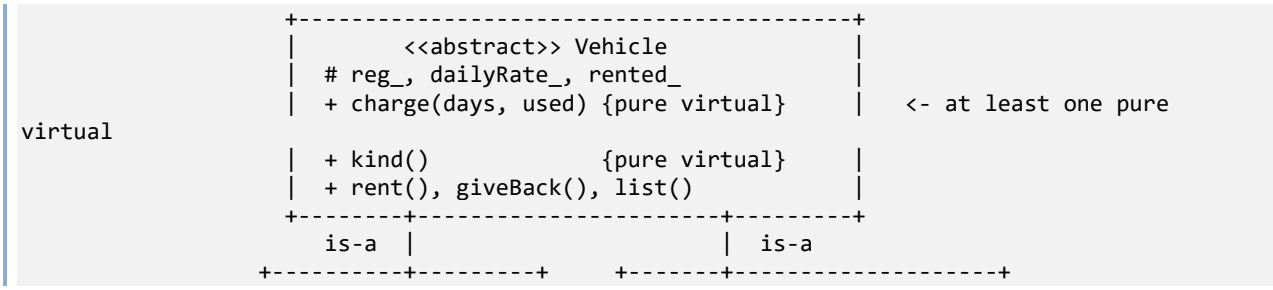
Stage 1 – Interpret the problem (2 marks)

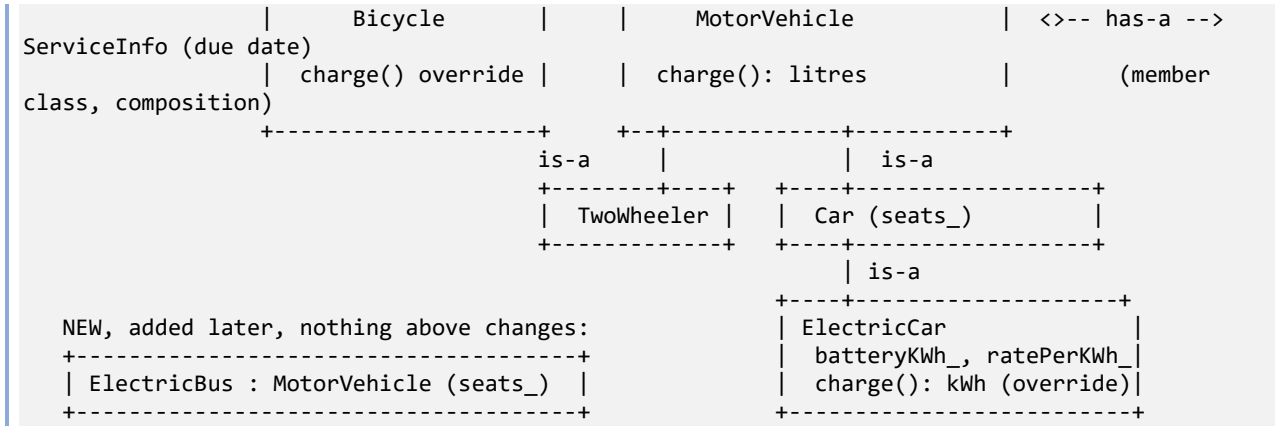
Entity	Data	Behaviour that differs
Vehicle (all)	registration number, daily rate, rented flag	rental charge
Bicycle	(nothing extra)	charge = days × daily rate
Motor vehicle (two-wheeler, car, electric car, bus)	fuel or charge level used, service due date	charge = days × rate + fuel used × price per litre
Car	seating capacity	same as motor vehicle
Electric car	battery capacity (kWh), price per kWh	charge = days × rate + kWh used × price per kWh (instead of litres)

Testable requirements

4. A bicycle rented for 3 days at Rs 100 is charged Rs 300.
5. A two-wheeler (Rs 400/day, Rs 100/litre) used 2 days, 5 litres, 1 day late is charged 2×400 + 5×100 + 1×400×1.5 = Rs 1,900.
6. An electric car (Rs 2,000/day, Rs 12/kWh) used 2 days, 40 kWh, 2 days late is charged 4,000 + 480 + 6,000 = Rs 10,480.
7. Renting a vehicle that is already rented, or returning one that is not rented, is rejected with an error and leaves the state unchanged.
8. A new vehicle type (e.g. an electric bus) can be added without editing any existing class.
9. All vehicles can be rented, returned and listed through one loop over Vehicle*.

Stage 2 – Propose the hierarchy (2 marks)





Types of inheritance used: single (Bicycle → Vehicle; TwoWheeler → MotorVehicle), multilevel (ElectricCar → Car → MotorVehicle → Vehicle) and hierarchical (Bicycle and MotorVehicle both derive from Vehicle). One member class: ServiceInfo inside MotorVehicle.

Stage 3 – Identify the critical design decisions (2 marks)

Decision	Choice and reason
What goes into Vehicle	Data every vehicle has (reg, daily rate, rented flag) and the common behaviour rent(), giveBack() (late-fee rule written once), list().
What goes into MotorVehicle	The service-due date (member class ServiceInfo) and the litre-based charge: only motor vehicles have them. Bicycle does not carry them.
Virtual functions	charge() (pure virtual in Vehicle), kind(), details(); a virtual destructor in Vehicle. These are the things that differ per type and must be chosen at run time.
Non-virtual	rent(), giveBack(), list(): the same algorithm for all types; giveBack() calls the virtual charge() (template-method style).
How the fleet is stored	An array of base-class pointers: Vehicle* fleet[N]. One loop handles all kinds; objects are not sliced; the array owns the objects and deletes them through the virtual destructor.
Errors	std::logic_error when renting a rented vehicle or returning a vehicle that is not rented.

Stage 4 – Design the classes (4 marks)

Pseudocode – rent and return a vehicle

```
ALGORITHM RentVehicle(v)
INPUT   v : pointer to Vehicle
OUTPUT  v marked as rented, or an error
BEGIN
   IF v.rented = TRUE THEN
        RAISE LogicError(v.reg + " is already rented")
   END IF
   v.rented <- TRUE
END

ALGORITHM ReturnVehicle(v, days, lateDays, used)
```

```
INPUT   v : pointer to Vehicle, days : rental days, lateDays : days after the due date,
        used : litres (or kWh) consumed
OUTPUT  amount due
BEGIN
   IF v.rented = FALSE THEN
        RAISE LogicError(v.reg + " was not rented")
   END IF
   v.rented <- FALSE
   base   <- v.charge(days, used)                // virtual call: Bicycle / Motor / Electric
rule
   late   <- lateDays * v.dailyRate * LATE_FEE_FACTOR
   RETURN base + late
END

ALGORITHM TotalRevenue(fleet, count, returns)
BEGIN
   total <- 0
   FOR i <- 0 TO count - 1 DO
        total <- total + ReturnVehicle(fleet[i], returns[i].days, returns[i].late,
returns[i].used)
   END FOR
   RETURN total
END
```

Class declarations (full working program below)

```
class Vehicle { protected: std::string reg_; double dailyRate_; bool rented_ = false;
  public: Vehicle(const std::string&, double); virtual ~Vehicle() = default;
          virtual double charge(int days, double used) const = 0;  virtual std::string kind()
const = 0;
          void rent(); double giveBack(int days, int lateDays, double used); void list()
const; };
class Bicycle      : public Vehicle      { /* charge = days * rate */ };
class MotorVehicle : public Vehicle      { ServiceInfo service_;  /* charge = days * rate +
litres * price */ };
class TwoWheeler   : public MotorVehicle { };
class Car          : public MotorVehicle { int seats_; };
class ElectricCar  : public Car          { double batteryKWh_, ratePerKWh_;  /* charge: kWh */
};
```

Stage 5 – Implement and demonstrate (3 marks)

Complete program (compiled with g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined)

```
// ===== Vehicle Rental Fleet =====  g++ -std=c++17 -Wall -Wextra
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
constexpr double FUEL_PRICE_PER_LITRE = 100.0;
constexpr double LATE_FEE_FACTOR = 1.5;                       // late day costs 1.5 x the
daily rate

class ServiceInfo {                                            // member class (has-a)
    std::string due_;
public:
    explicit ServiceInfo(const std::string& due) : due_(due) {}
    const std::string& due() const { return due_; }
};
class Vehicle {                                                // abstract base
protected:
```

```
    std::string reg_; double dailyRate_; bool rented_ = false;
public:
    Vehicle(const std::string& reg, double rate) : reg_(reg), dailyRate_(rate) {}
    virtual ~Vehicle() = default;
    virtual double charge(int days, double used) const = 0;    // pure virtual: differs per
type
    virtual std::string kind() const = 0;
    virtual std::string details() const { return ""; }
    void rent() { if (rented_) throw std::logic_error(reg_ + " is already rented"); rented_ =
true; }
    double giveBack(int days, int lateDays, double used) {     // template method: common rule
written once
        if (!rented_) throw std::logic_error(reg_ + " was not rented");
        rented_ = false; return charge(days, used) + lateDays * dailyRate_ * LATE_FEE_FACTOR;
}
    bool rented() const { return rented_; }
    const std::string& reg() const { return reg_; }
    void list() const { std::cout << std::left << std::setw(14) << kind() << std::setw(8) <<
reg_ << std::setw(8) << (rented_ ? "RENTED" : "free") << details() << '\n'; }
};
class Bicycle : public Vehicle {
public:
    using Vehicle::Vehicle;
    double charge(int days, double) const override { return days * dailyRate_; }
    std::string kind() const override { return "Bicycle"; }
};
class MotorVehicle : public Vehicle {                          // fuel/charge level and
service due date
protected:
    ServiceInfo service_;
public:
    MotorVehicle(const std::string& reg, double rate, const std::string& due) : Vehicle(reg,
rate), service_(due) {}
    double charge(int days, double litres) const override { return days * dailyRate_ + litres
* FUEL_PRICE_PER_LITRE; }
    std::string details() const override { return "service due " + service_.due(); }
};
class TwoWheeler : public MotorVehicle { public: using MotorVehicle::MotorVehicle; std::string
kind() const override { return "TwoWheeler"; } };
class Car : public MotorVehicle {
protected:
    int seats_;
public:
    Car(const std::string& reg, double rate, const std::string& due, int seats) :
MotorVehicle(reg, rate, due), seats_(seats) {}
    std::string kind() const override { return "Car"; }
    std::string details() const override { return MotorVehicle::details() + ", " +
std::to_string(seats_) + " seats"; }
};
class ElectricCar : public Car {                               // charged per kWh instead of
per litre
    double batteryKWh_, ratePerKWh_;
public:
    ElectricCar(const std::string& reg, double rate, const std::string& due, int seats, double
batt, double perKWh)
        : Car(reg, rate, due, seats), batteryKWh_(batt), ratePerKWh_(perKWh) {}
    double charge(int days, double kWh) const override { return days * dailyRate_ + kWh *
ratePerKWh_; }
    std::string kind() const override { return "ElectricCar"; }
    std::string details() const override { return Car::details() + ", battery " +
std::to_string((int)batteryKWh_) + " kWh"; }
};
// ---- NEW vehicle type added later, in new code only: no existing class is edited ----
```

```
class ElectricBus : public MotorVehicle {
    int seats_; double ratePerKWh_;
public:
    ElectricBus(const std::string& reg, double rate, const std::string& due, int seats, double
perKWh) : MotorVehicle(reg, rate, due), seats_(seats), ratePerKWh_(perKWh) {}
    double charge(int days, double kWh) const override { return days * dailyRate_ + kWh *
ratePerKWh_; }
    std::string kind() const override { return "ElectricBus"; }
    std::string details() const override { return MotorVehicle::details() + ", " +
std::to_string(seats_) + " seats"; }
};
int main() {
    Vehicle* fleet[6] = { new Bicycle("B-101", 100), new TwoWheeler("TW-202", 400, "2026-12-
01"), new Car("C-303", 1500, "2026-11-20", 5),
                          new ElectricCar("EV-404", 2000, "2027-01-15", 5, 60, 12.0), nullptr,
nullptr };
    fleet[4] = new ElectricBus("EB-505", 6000, "2027-02-01", 30, 12.0);          // added
without touching existing classes
    int count = 5;
    std::cout << "--- fleet ---\n"; for (int i = 0; i < count; ++i) fleet[i]->list();
    for (int i = 0; i < count; ++i) fleet[i]->rent();
    std::cout << "--- after renting all ---\n"; for (int i = 0; i < count; ++i) fleet[i]-
>list();
    struct Ret { int days, late; double used; } r[] = { {3, 0, 0}, {2, 1, 5}, {4, 0, 30}, {2,
2, 40}, {1, 0, 150} };
    double total = 0; std::cout << std::fixed << std::setprecision(2) << "--- returns (days,
late days, litres or kWh used) ---\n";
    for (int i = 0; i < count; ++i) { double due = fleet[i]->giveBack(r[i].days, r[i].late,
r[i].used); total += due; std::cout << std::left << std::setw(14) << fleet[i]->kind() <<
std::setw(8) << fleet[i]->reg() << std::right << std::setw(10) << due << '\n'; }
    std::cout << "Total revenue " << total << "   (hand check: 300 + 1900 + 9000 + 10480 +
7800 = 29480)\n";
    try { fleet[0]->giveBack(1, 0, 0); } catch (const std::logic_error& e) { std::cout <<
"error handled: " << e.what() << "\n"; }
    try { fleet[1]->rent(); fleet[1]->rent(); } catch (const std::logic_error& e) { std::cout
<< "error handled: " << e.what() << "\n"; }
    for (int i = 0; i < count; ++i) delete fleet[i];
    std::cout << "sizeof: Bicycle=" << sizeof(Bicycle) << " TwoWheeler=" << sizeof(TwoWheeler)
<< " Car=" << sizeof(Car) << " ElectricCar=" << sizeof(ElectricCar) << " ElectricBus=" <<
sizeof(ElectricBus) << "\n";
}
```

Actual output

```
--- fleet ---
Bicycle       B-101   free
TwoWheeler    TW-202  free    service due 2026-12-01
Car           C-303   free    service due 2026-11-20, 5 seats
ElectricCar   EV-404  free    service due 2027-01-15, 5 seats, battery 60 kWh
ElectricBus   EB-505  free    service due 2027-02-01, 30 seats          <- the new type, added
without editing old classes
--- after renting all ---                                              (all five show RENTED)
--- returns (days, late days, litres or kWh used) ---
Bicycle       B-101       300.00
TwoWheeler    TW-202     1900.00
Car           C-303      9000.00
ElectricCar   EV-404    10480.00
ElectricBus   EB-505     7800.00
Total revenue 29480.00   (hand check: 300 + 1900 + 9000 + 10480 + 7800 = 29480)
error handled: B-101 was not rented
error handled: TW-202 is already rented
sizeof: Bicycle=56 TwoWheeler=88 Car=96 ElectricCar=112 ElectricBus=104
```

Hand check: Car 4 × 1500 + 30 × 100 = 9,000; ElectricBus 1 × 6000 + 150 × 12 = 7,800; the others as in the requirements. The run under AddressSanitizer and UndefinedBehaviorSanitizer reported nothing (exit code 0), so every object was destroyed through the virtual destructor. The extension: ElectricBus is a new class in new code; Vehicle, MotorVehicle, Car and the others were not edited, and the existing loop handled it unchanged.

Alternative concept – one Vehicle class with a type code and a switch in charge()

Measure	Abstract base + virtual functions	Single class + type code + switch
Time per charge() call (10,000,000 calls, -O2, measured)	O(1): 108 ms (indirect call through the vtable)	O(1): 130 ms (switch); the two are the same order and the difference is dominated by memory access; totals agreed
Space per object	one vtable pointer (8 bytes) + only the fields that type needs: Bicycle 56 bytes up to ElectricCar 112	every object carries every field (reg, seats, battery, per-kWh rate, service date …): 112 bytes even for a bicycle
Classes/places changed to add an electric bus	0 existing classes edited; 1 new class	Add a type code; edit the switch in charge() and every other switch (kind, details); edit the constructor – all existing code is touched and re-tested
Safety when a case is forgotten	Pure virtual charge(): the program does not compile until it is written	A missing case silently returns 0 (charge 0.00) unless a -Wswitch warning is noticed

Simulation Screenshot

Real terminal run: the repaired payroll hierarchy, the Task 2 and Task 3 predictions, the defective and repaired account programs (including the AddressSanitizer error), the Cart program and the vehicle-rental fleet.

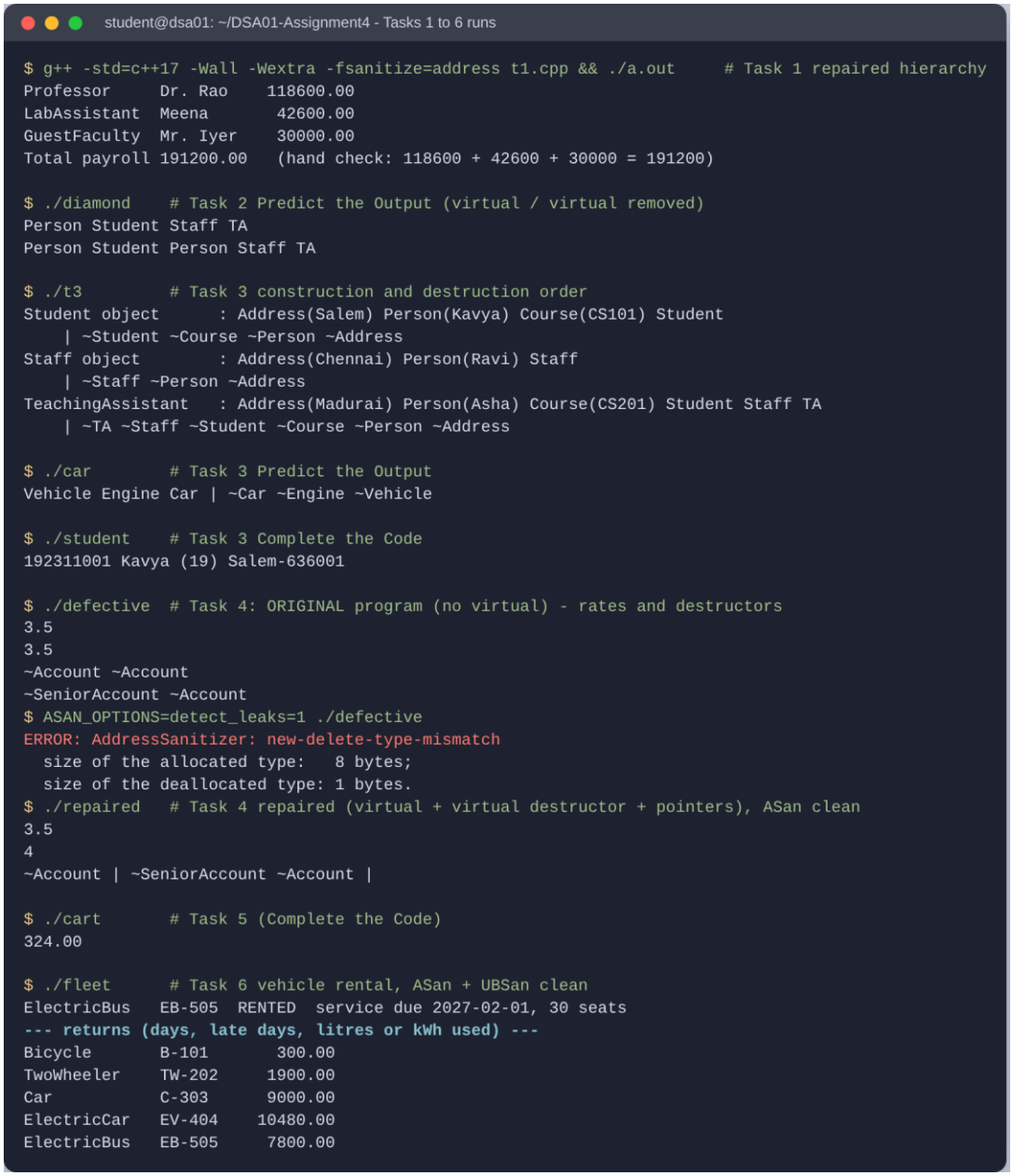


Figure 1 – Terminal run of the Assignment 4 programs.